Project Title: AI FIT TRACK

Team ID : SWTID-2026-5429

Team Size : 4

Team Leader : ELAKKIYA.B &

elakkiyaboopalan3112@gmail.com

Team member : ENIYA.K & eniyak768@gmail.com

Team member : DURGAPARAMESHWARI.D &

durgadhandappani07@gmail.com

Team member : ELAVARASI.M &

elavarasielavarasimahimahendra@gmail.com

INTRODUCTION

In the contemporary digital era, personal health management and physical fitness have emerged as paramount priorities for individuals striving to maintain balanced, healthy lifestyles. Despite growing public interest in physical wellness, maintaining consistent exercise regimens remains a significant challenge. Many individuals struggle with fragmented tracking mechanisms, subjective exercise planning, and an absence of tailored professional instruction. Conventional fitness recording relies heavily on manual physical logs, unstructured digital notes, or static spreadsheets, resulting in dispersed records, lost longitudinal patterns, and human error.

The integration of modern software architectures and generative artificial intelligence offers a transformative paradigm shift in physical conditioning. Automated computational platforms can systematically track exercise metrics while converting static data points into dynamic behavioral feedback.

FitTrack AI is developed as a secure, enterprise-grade, intelligent backend infrastructure designed to resolve these traditional bottlenecks. Utilizing a decoupled Application Programming Interface design, the system delivers automated record management, role-based identity validation, and customized workout regimens powered by large language models. By centralizing exercise data and abstracting complex training methodologies, the platform provides scalable, context-aware athletic management suitable for individuals, fitness trainers, and enterprise digital wellness environments.

PROJECT OVERVIEW

FitTrack AI is an artificial intelligence-assisted RESTful backend system constructed to support end-to-end workout logging, administrative exercise management, and personalized health analytics. The architecture decouples client-side interfaces from data processing mechanisms, relying on standardized JSON interactions to support consumer clients across web browsers, cross-platform mobile environments, and automated administrative dashboards.

The system comprises four core subsystems:

- Identity and Access Management Subsystem: Implements token-based user verification, credential salting and hashing, and route protection shields to maintain strict boundaries across isolated user partitions.

- Workout Persistence Engine: Governs document models for individual exercise events, validating attributes including volume, movement classifications, duration, and caloric calculations.
- Analytical Query Core: Provides indexed, multi-parameter search algorithms capable of isolating records via exercise categories, user identity keys, or date bounds.
- Cognitive Intelligence Subsystem: Employs Google Gemini AI via prompt-engineered orchestration to synthesize workout variables into personalized routines and statistical assessments.

A common operational case study involves athletes or fitness enthusiasts who consistently exercise but maintain their logs through physical notebooks or personal spreadsheets. This manual pattern introduces calculation errors, prevents historical cross-referencing, and lacks real-time guidance. FitTrack AI bridges this operational gap by replacing dispersed logs with an integrated backend ecosystem that safeguards user privacy while generating data-driven exercise schedules.

OBJECTIVES AND PROBLEM STATEMENT

Problem Statement: Athletes, fitness enthusiasts, and everyday gym users encounter systemic operational frictions when tracking physical conditioning without specialized tools:

- Data Fragmentation and Inaccuracy: Relying on physical journals or disconnected electronic spreadsheets scatters workout history, leading to incomplete records, omitted sessions, and calculation discrepancies.
- Absence of Identity Isolation: Standard document trackers lack multi-user security boundaries, leaving sensitive user wellness metrics exposed without data isolation.
- High Query Latency: Manually sifting through unstructured workout entries makes chronological or category-based lookups inefficient.
- Generic Training Recommendations: Non-contextual fitness plans fail to account for a user's chronological age, existing experience, or targeted wellness objectives, increasing the risk of overtraining or injury.
- Lack of Performance Analytics: Users frequently log exercise durations and estimated caloric expenditures without structured feedback to contextualize sustained training progress.

Primary Objectives:

- Deliver a Centralized, Decoupled Architecture: Build a service-oriented backend exposing standardized RESTful endpoints that decouple database mutations from client rendering layers.
- Enforce Zero-Trust Data Isolation: Protect proprietary user records by combining client-side JSON Web Token exchanges with server-side bcrypt cryptographic hashing.
- Automate Dynamic Document Lifecycle Operations: Enable full CRUD (Create, Read, Update, Delete) data operations over normalized workout collections with automated schema validation.
- Provide Indexed Query Capabilities: Allow efficient record retrieval through parameterized filters targeting workout identifiers, exercise typologies, and timestamp bounds.
- Integrate Generative Fitness Synthesis: Automate the production of structured exercise plans and performance evaluations by integrating the Google Gemini generative model via dedicated service wrappers.

TECHNOLOGY USED

Core Frameworks and Runtime:

- Node.js (LTS Runtime): An asynchronous, event-driven JavaScript engine that uses non-blocking I/O abstractions to handle concurrent HTTP connections efficiently under heavy request loads.
- Express.js: A minimalist server framework running on Node.js that establishes structured routing pipelines, cross-cutting middleware chains, and JSON body parsing.

Database and Data Modeling:

- MongoDB: A NoSQL document database that persists schema-flexible, BSON-encoded data records, allowing dynamic entity modeling and rapid horizontal read-write scaling.
- Mongoose ODM: An Object Data Modeling library that provides schema definitions, type casting, pre-save hooks, field validations, and relational mapping within the application layer.

Security and Middleware:

- JSON Web Tokens (JWT): A stateless authentication standard that encodes cryptographically signed user payloads into compact strings, verifying incoming user requests without persistent server sessions.
- Bcrypt.js: An adaptive cryptographic hashing library that generates salted hashes from plaintext passwords, safeguarding stored authentication credentials.
- CORS (Cross-Origin Resource Sharing): Security middleware that restricts cross-domain communication, ensuring API endpoints accept requests exclusively from authorized client origins.
- Dotenv: An environment isolation module that manages runtime variables, shielding database connection strings and AI secrets from code repositories.

Artificial Intelligence:

- Google Gemini AI Engine: A multimodal large language model accessed through API service abstractions, transforming structured user profiles and cumulative workout metrics into contextual training schedules and analytical feedback.

Development and Testing Suite:

- Postman and VS Code ThunderClient: API testing and assertion tools used to evaluate HTTP status codes, payload structures, authentication headers, and database updates.

Technology Stack Distribution:

- Server Runtime: Node.js (handles asynchronous I/O execution)
- Web Framework: Express.js (manages HTTP routing and middleware pipelines)
- Database Engine: MongoDB (handles document-oriented data storage)
- Data Modeling: Mongoose ODM (governs schema structure and business validation)
- Security Layer: JWT and Bcrypt.js (handle authentication tokens and password encryption)
- Intelligence Engine: Google Gemini AI (delivers context-aware recommendations)
- Validation Suite: Postman / ThunderClient (validates end-to-end API communication)

SYSTEM REQUIREMENTS AND METHODOLOGY

Hardware Specifications:

- Central Processing Unit: Intel Core i5 (8th Generation or above), AMD Ryzen 5, or equivalent multi-core processor.
- Random Access Memory: 8 GB minimum, with 16 GB recommended for handling concurrent database instances and backend services.
- Storage Workspace: At least 1 GB of available disk workspace for project dependencies, logs, and development libraries.

Software Specifications:

- Operating System: Cross-platform support across Microsoft Windows 10/11, macOS, and Linux distributions.
- Runtime Environment: Node.js version 16.0.0 or higher.
- Package Management: Node Package Manager (NPM) version 8.0.0 or higher.
- Database Management System: MongoDB Community Server (local) or MongoDB Atlas cloud deployment.
- API Client: Postman desktop software or VS Code ThunderClient extension.

Architectural Methodology: Layered MVC Pattern FitTrack AI organizes operations using the Model-View-Controller architectural pattern, separating concerns across distinct system boundaries:

- Client Presentation Layer: While traditional web apps render server-side HTML, this backend architecture treats external API consumers (React interfaces, native mobile apps, Postman) as the view tier. The client initiates HTTP requests, stores received JWT bearer tokens, and displays parsed JSON data.
- Server Gateway and Middleware: The Express server parses incoming payloads and passes them through an authentication middleware chain. This middleware inspects the HTTP Authorization header, verifies token signatures, decodes user credentials, and rejects unauthenticated requests before they reach the controller.

- Controller Business Logic Layer: Controllers decouple incoming requests from database operations. They validate incoming attributes, call relevant Mongoose database routines, dispatch prompts to the Gemini service wrapper, and return standardized JSON responses.
- Model and Data Layer: Mongoose models handle data persistence. They define schemas, enforce data types, execute cryptographic hooks prior to saving, and maintain relational references between user accounts and workout entries.
- External Service Layer: Dedicated service modules isolate external API logic, specifically managing payload packaging, network timeouts, and response extraction for Google Gemini AI.

WORKING OF THE SYSTEM

Step-by-Step Operational Workflow:

Step 1: User Registration and Cryptographic Processing A new user registers by submitting their name, email address, and plaintext password. The authentication controller checks whether the email already exists in MongoDB. If unique, the user schema triggers a pre-save hook using bcrypt.js to generate an encrypted salt and hash of the password. The sanitized document is stored, and the server generates a signed JSON Web Token.

Step 2: Credential Verification and Token Issuance During login, the user submits their registered email and password. The system searches MongoDB for a matching email record. If found, it compares the submitted password against the stored bcrypt hash. Once verified, the server issues a signed JWT containing the user ID, which the client retains for subsequent requests.

Step 3: Route Protection and Request Interception When making requests to protected endpoints, the client includes the JWT in the Authorization Bearer header. Middleware intercepts the request, verifies the token's cryptographic integrity, extracts the user ID, and attaches it directly to the request object. Requests lacking a valid token are denied access immediately.

Step 4: Exercise Document Processing To log a workout, the authenticated user submits details such as the exercise name, category, duration, calories burned, and date. The controller verifies

these fields against schema rules (e.g., non-negative calorie counts, minimum one-minute durations). Once validated, the workout is linked to the user's ID and stored in MongoDB.

Step 5: Query Execution and Historical Retrieval Users can query their saved workouts by ID, category, or date range. The controller queries MongoDB specifically for records matching the authenticated user's ID, ensuring multi-tenant data isolation, and returns the records alongside total session counts.

Step 6: AI-Assisted Recommendation Generation When an AI recommendation is requested, the user submits profile data such as age, fitness goals, and experience level. The Gemini service compiles these inputs into a structured prompt. Google Gemini processes the prompt and returns a personalized weekly exercise plan, target movement suggestions, and safety precautions in a structured JSON payload.

Step 7: AI Fitness Insight Synthesis To receive performance evaluations, the client submits cumulative metrics, including total completed sessions, average workout durations, and total calories burned. The AI service evaluates these historical numbers and returns behavioral feedback, progress milestones, and suggestions for progressive overload.

IMPLEMENTATION AND OUTPUT

Data Schema Architecture:

User Schema Model

- Collection Name: users
- Primary Key Identifier: ObjectId
- Name Field: Text string, mandatory input, whitespace trimmed.
- Email Field: Text string, mandatory input, strictly unique index, whitespace trimmed, converted to lowercase characters.
- Password Field: Text string, mandatory input, minimum length enforced at six characters.
- Timestamps: Automatically generated creation and modification dates.
- Security Hooks: Pre-save middleware executing bcrypt encryption with ten salt rounds; custom method for comparing entered passwords with stored hashes.

Workout Schema Model

- Collection Name: workouts
- Primary Key Identifier: ObjectId
- User Association: Mandatory ObjectId referencing the primary key of the users collection.
- Workout Name: Text string, mandatory input, whitespace trimmed.
- Category: Text string, restricted to enumerated values including Cardio, Strength Training, Yoga, Running, Cycling, and Walking.
- Duration: Numeric value, mandatory, minimum threshold set at one minute.
- Calories Burned: Numeric value, mandatory, minimum threshold set at zero.
- Workout Date: Date value, defaults to the current system timestamp if omitted.
- Timestamps: Automatically managed creation and modification dates.

RESTful Interface Endpoints:

- POST /api/auth/register: Public endpoint for creating user accounts and issuing initial JWTs.
- POST /api/auth/login: Public endpoint for verifying user credentials and issuing JWTs.
- GET /api/auth/profile: Protected endpoint for retrieving user profile data using bearer token authentication.
- POST /api/workouts/: Protected endpoint for saving a new workout record linked to the user ID.
- GET /api/workouts/: Protected endpoint for retrieving all workout entries associated with the user.
- GET /api/workouts/:id: Protected endpoint for fetching a specific workout record by its unique ID.
- PUT /api/workouts/:id: Protected endpoint for updating an existing workout record.
- DELETE /api/workouts/:id: Protected endpoint for permanently removing a workout record.

- POST /api/ai/workout-recommendation: Protected endpoint for generating personalized workout plans via Gemini AI.
- POST /api/ai/fitness-insights: Protected endpoint for generating training progress summaries via Gemini AI.

Output Verification and Transaction Profiles:

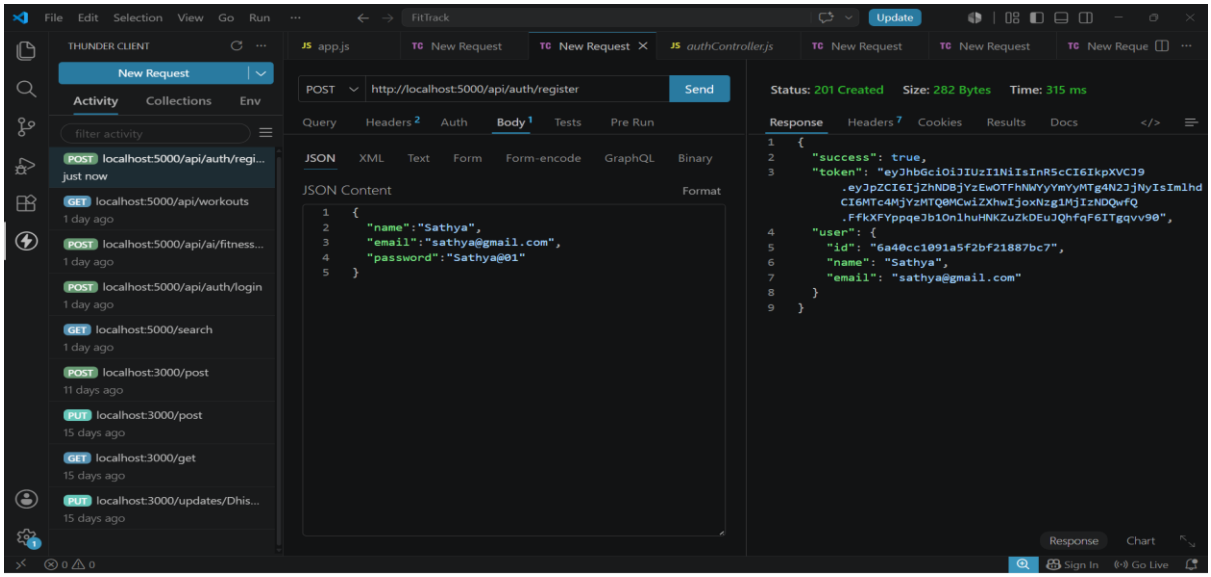
- User Registration Execution: Target Endpoint: POST /api/auth/register Submission Data: Name as Sathya, Email as sathya@gmail.com, Password as Sathya@01 Response Status: 201 Created Returned Data: Success status flag set to true, generated JWT bearer string, and created user profile containing unique database ID, name, and email.
- User Login Execution: Target Endpoint: POST /api/auth/login Submission Data: Email as sathya@gmail.com, Password as Sathya@01 Response Status: 200 OK Returned Data: Success status flag set to true, active JWT bearer string, and verified user profile details.
- User Profile Retrieval: Target Endpoint: GET /api/auth/profile Header Authorization: Bearer JWT Token Response Status: 200 OK Returned Data: Authenticated user details including primary account ID, user name, and registered email address.
- Workout Creation: Target Endpoint: POST /api/workouts/ Header Authorization: Bearer JWT Token Submission Data: Workout Name as Evening Jog, Category as Running, Duration as 30, Calories Burned as 350, Workout Date as 2026-06-26 Response Status: 201 Created Returned Data: Success status flag set to true, accompanied by the created document containing the generated ID, the user foreign key, and creation timestamps.
- Workout Retrieval: Target Endpoint: GET /api/workouts/ Header Authorization: Bearer JWT Token Response Status: 200 OK Returned Data: Success status flag set to true, workout count equal to 1, and an array containing the Evening Jog document.
- Single Record Lookup: Target Endpoint: GET /api/workouts/:id Header Authorization: Bearer JWT Token Response Status: 200 OK Returned Data: Retrieves the single workout document matching the requested ID.
- Workout Modification: Target Endpoint: PUT /api/workouts/:id Header Authorization: Bearer JWT Token Submission Data: Modified Duration as 60, Modified Calories

Burned as 650 Response Status: 200 OK Returned Data: Updated workout document confirming the new duration and calorie values, alongside an updated modification timestamp.

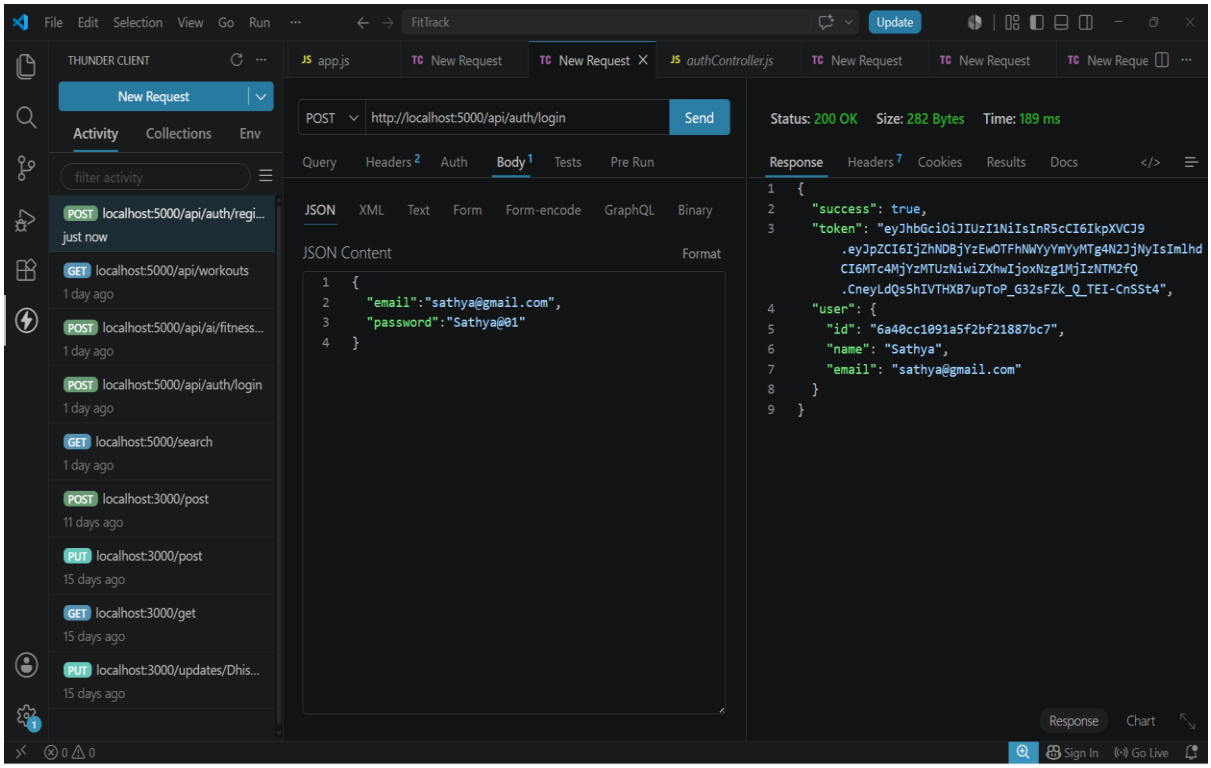
- Workout Deletion: Target Endpoint: DELETE /api/workouts/:id Header Authorization: Bearer JWT Token Response Status: 200 OK Returned Data: Success status flag set to true, accompanied by confirmation that the workout was successfully removed.
- AI Workout Recommendation Generation: Target Endpoint: POST /api/ai/workout-recommendation Header Authorization: Bearer JWT Token Submission Data: Age as 22, Fitness Goal as Weight Loss, Experience as Beginner Response Status: 200 OK Returned Data: A personalized training plan advising three weekly sessions of combined cardiovascular exercise and full-body bodyweight movements (squats, knee push-ups, planks), complete with warm-up and cool-down routines.
- AI Fitness Insights Generation: Target Endpoint: POST /api/ai/fitness-insights Header Authorization: Bearer JWT Token Submission Data: Total Workouts as 18, Average Duration as 45, Total Calories Burned as 6200 Response Status: 200 OK Returned Data: An analytical assessment evaluating the user's exercise consistency, highlighting steady progress across sessions, and suggesting gradual increases in training intensity.

Authentication API Testing

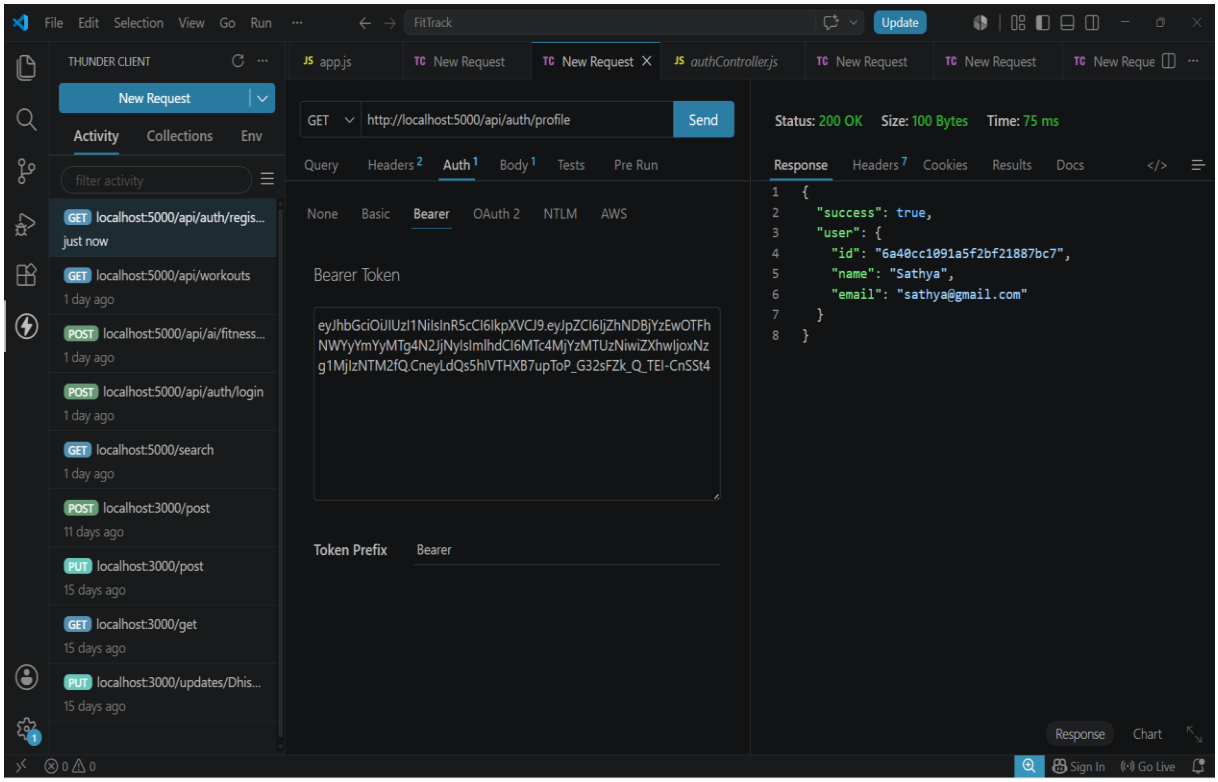
Register User



Login User

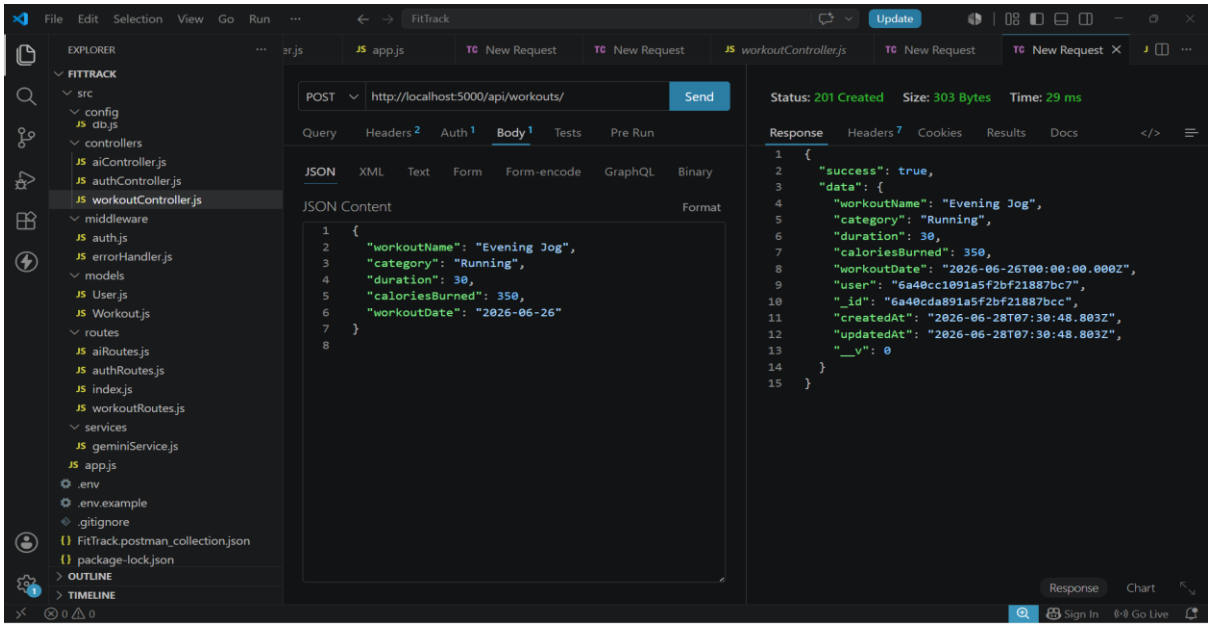


Get Profile

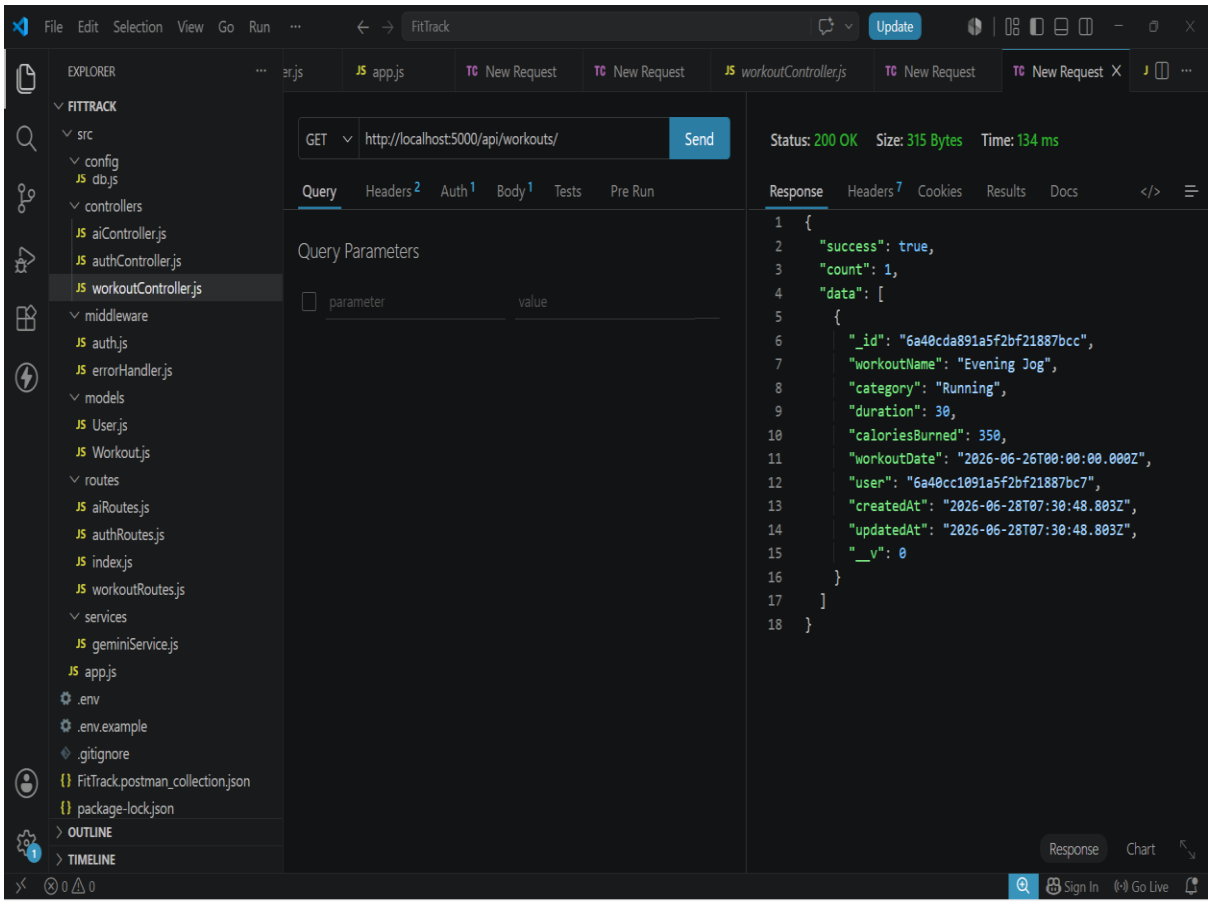


Workout API Testing

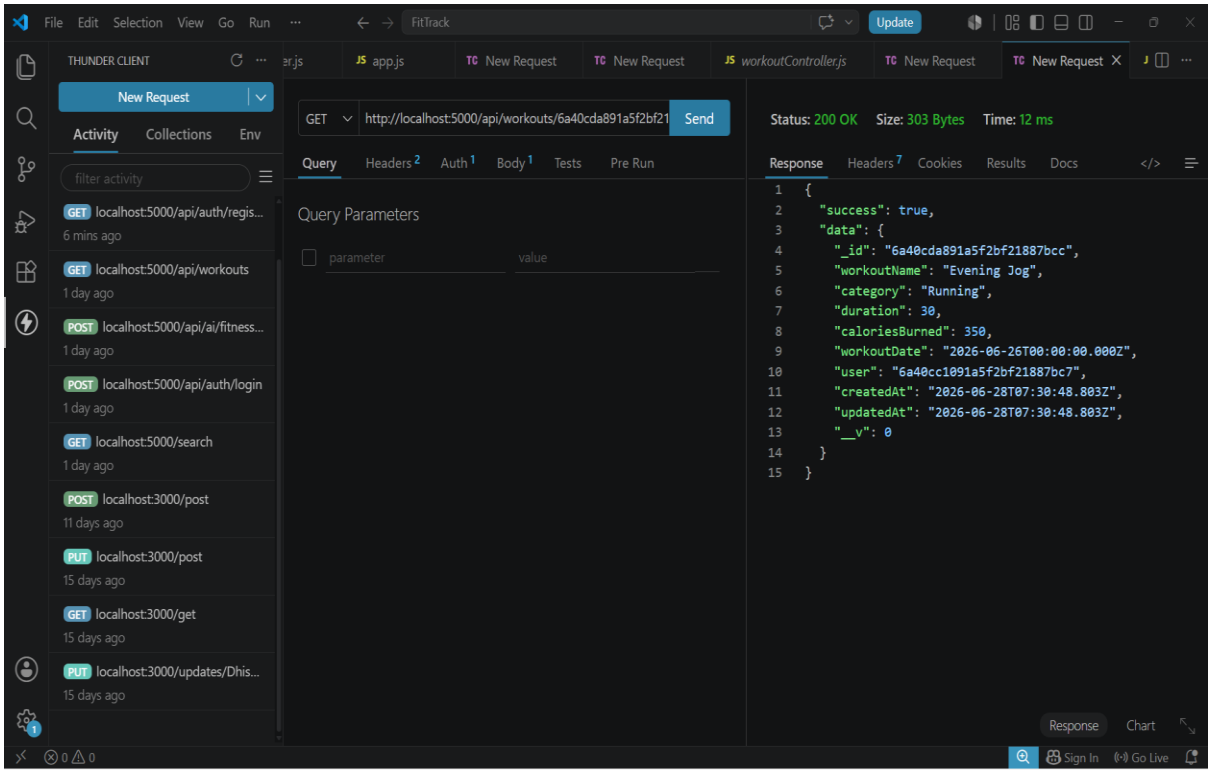
Add Workout



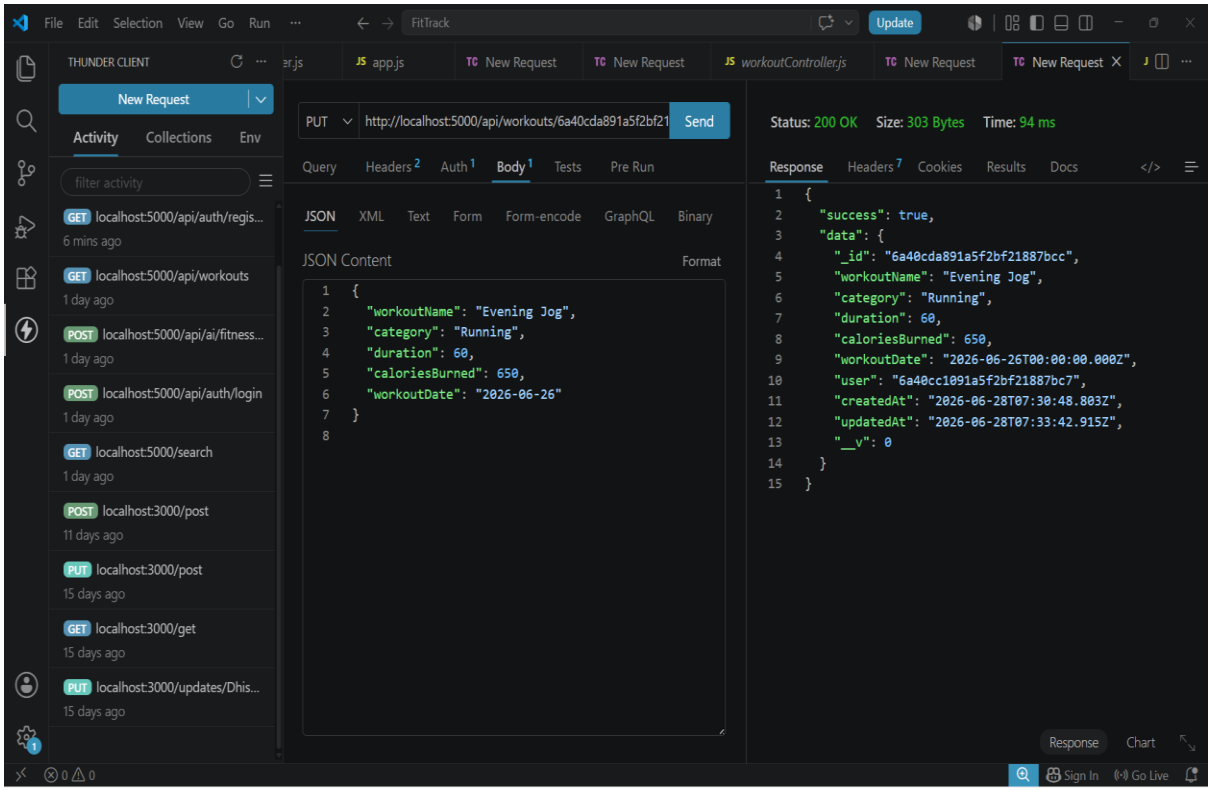
Get All Workouts



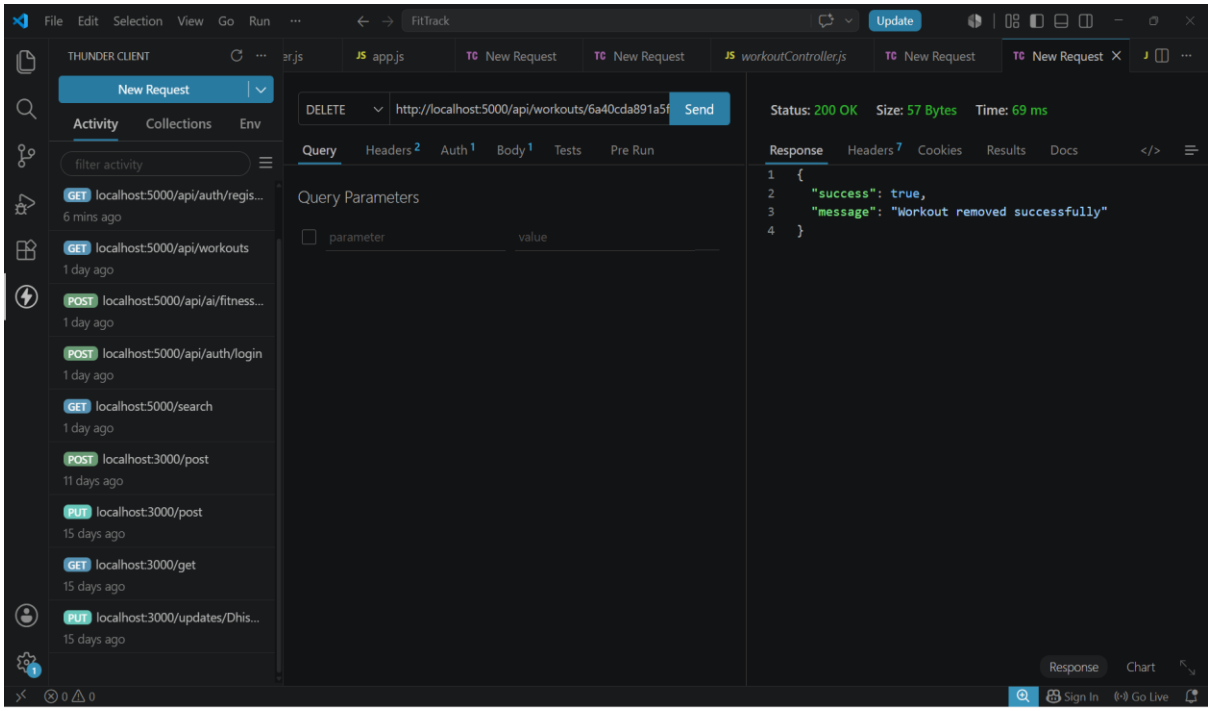
Get Workout By ID



Update Workout

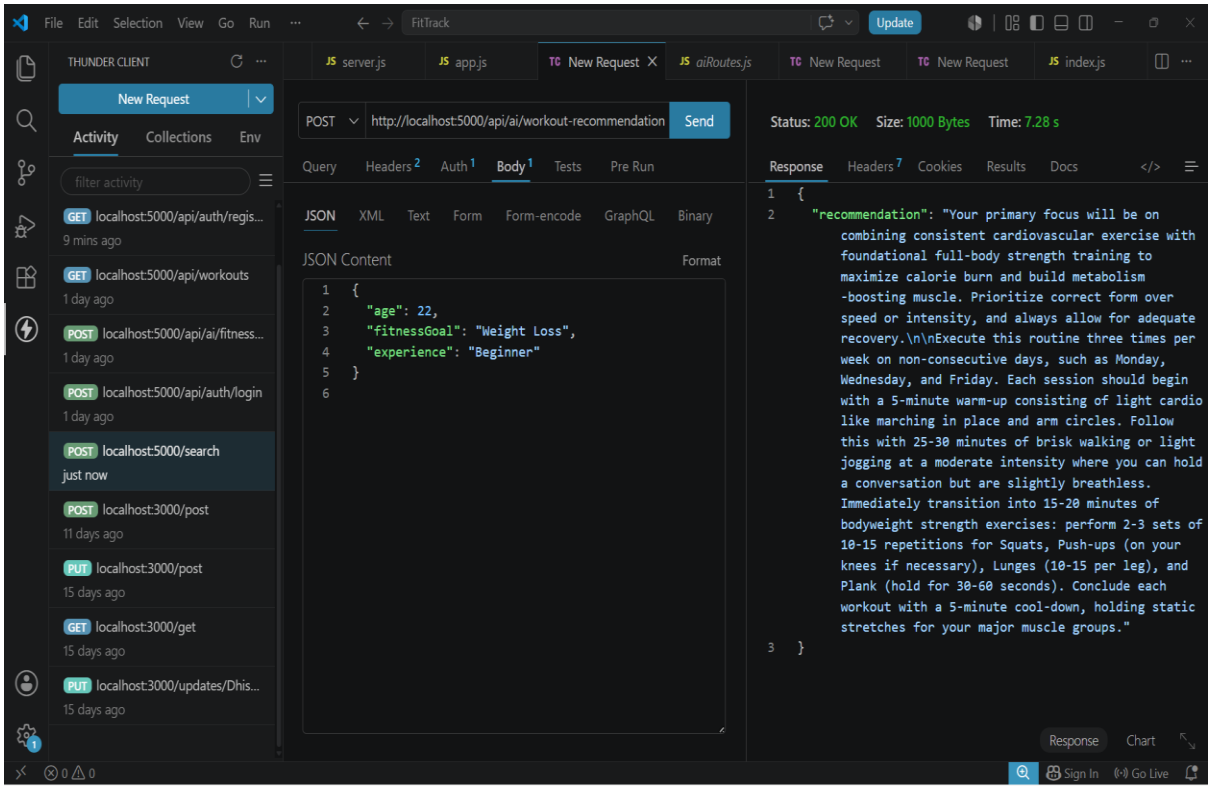


Delete Workout

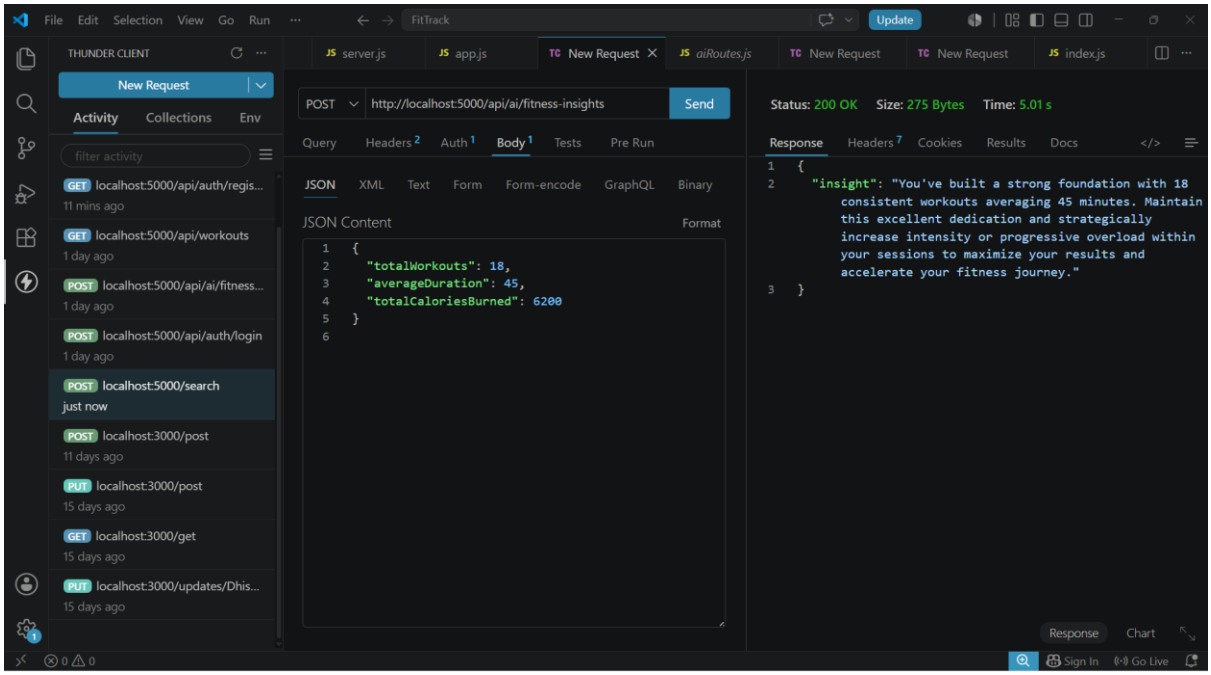


AI API Testing

Workout Recommendation



Fitness Insights



APPLICATIONS AND ADVANTAGES

Real-World Applications:

- Personalized Digital Fitness Platforms: Functions as a modular backend engine for consumer-facing mobile and web wellness applications.
- Gym and Athletic Facility Management: Provides commercial fitness centers and personal trainers with automated tracking tools to monitor client training history.
- Corporate Wellness Portals: Integrates with enterprise platforms to support employee fitness initiatives and track aggregated wellness activities.
- Rehabilitation and Physical Therapy Tracking: Enables physical therapy clinics to monitor low-impact exercise routines and review recovery timelines.

Key Technical Advantages:

- Modular Architectural Separation: Using MVC principles to isolate routes, controllers, middleware, and models simplifies maintenance, unit testing, and collaborative development.

- Context-Aware Adaptive Intelligence: Unlike static fitness apps that offer rigid templates, FitTrack AI uses Google Gemini to generate dynamic workouts matched to user age, background, and performance data.
- Stateless Security and Scalability: JSON Web Tokens eliminate server-side session tracking, reducing memory overhead and allowing straightforward horizontal scaling behind load balancers.
- Centralized Data Hygiene and Integrity: Mongoose schemas enforce data validation, field restrictions, and type checks, preventing malformed or invalid records from entering the database.
- Client-Agnostic Interface Design: Standardized JSON formatting enables seamless integration with diverse frontends, including React, Angular, Vue, React Native, iOS, and Android.

CONCLUSION

FitTrack AI successfully resolves the limitations of manual fitness tracking by providing a robust, scalable backend for managing physical training routines. By pairing the lightweight efficiency of Node.js and Express with the flexible document storage of MongoDB, the platform delivers secure, low-latency API access for multi-user wellness environments. Integrating Google Gemini AI bridges the gap between raw data collection and actionable coaching, turning passive exercise logs into customized, safety-conscious training recommendations. The resulting system serves as a maintainable, high-performance foundation capable of powering modern digital health applications.